NAME:

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Friday Oct. 23rd by the end of class.

Portfolio 6: Strings

To succeed in these problems you will need to read and engage with the embedded activities in the **Runestone texbook chapter(s)** (worth 0.5 of a grade percentage point):

- 8: Sequences
- 9: Transforming Sequences (to prepare for PF 7)

Note that these activities are focused on this week's theme, but they are designed to utlize skills from all your previous portfolios as well. You should use your judgement to decide which tasks are best solved with which skills you have accumulated so far.

For completing and correcting 70% of the (A) activities, you will earn 1 grade percentage point. For participating in a *share-out of 1 of the activities, you will earn 0.5 of a grade percentage point*.

Note that some activities we will plan to start in class and others you need to start on your own. You should finish each activity before its in-class share-out in order to correct it before final submission.

Submission: To get credit for completing a prgramming activity, submit it to gradescope and answer the written questions about your algorithm, whether your program worked and what troubleshooting you can think of. You *only have to print programs you want my feedback on*. Printed programs can still be marked up by hand for corrections.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out:

Part	Problem/Task	Page	In-class Start	In-class Share-out	Submission Format
A1	Written prompt	2	F 10/16	F 10/16	written
A2	Dev. as Dec. Makers (reading)	3	oyo	M 10/19	written
A3	Dev. as Dec. Makers (report)	4	M 10/19	M 10/19	written, gdoc
A4	Dev. as Dec. Makers (reflection)	5	oyo	W 10/21	written
A5	Prog. with Pytthon (ascii strings)	7	W 10/21	W 10/216	GS, print
A6	Prog. with Pytthon (binary)	8	oyo	Th 10/22	GS, print
A7	Prog. with Pytthon (Pig Latin)	9	oyo	Th 10/22	GS, print
A8	Prog. with Pytthon (caesar cipher)	10	Th 10/22	F 10/23	GS, print
A9	Written Questions	11	oyo	F 10/23	written
A10	Prog. with Pytthon (breakcode)	13	oyo	F 10/23	GS, print
B1	Share-out report	15	oyo	written	written
B2	Share-out report (optional)	16	oyo	written	written

A1) Prompt: Is using an algorithm or human judgement more "fair" (unbiased) when awarding assets such as preferred student housing?

A2) Developers as Decision Makers: background reading

1. Read through the acticity description
2. Each group member should read (at least) 1 of the following background readings to summarize for the group

    - Silicon Valley Triage Tool
    - Economic Roundtable on the Silicon Valley Triage Tool
    - A healthcare algorithm started cutting care
    - We created poverty

3. Who in your group is reading which? Circle what you will read.

    .....

    .....

    .....

    .....

4. What stood out to you in your reading? What do you want to share with your group about it?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

A3) Developers as Decision Makers: results

1. Make a group copy of the Google Doc worksheet template and share it with all group members
2. Edit your worksheet as you plan, implement, test and reflect upon your team's housing algorithm
    - – example housing lottery
3. Do you expect your algorithm to produce different results than other groups in the class? Why or why not?

    .....

    .....

    .....

    .....

    .....

    .....
4. Record the results from the different groups (draw a plot/graph, record numbers, state results)

A4) Developers as Decision Makers: reflection

1. Did your algorithm to produce different results than other groups in the class?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

2. Did any algorithms from other groups stand out? Why or why not?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. Did any algorithms seem "fair"? Which one seemed most fair? Why or why not?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

A5) ASCII and Characters

Learning goals / relevent topics

1. ASCII encoding of strings (https://www.asciitable.com/)

Write a function that takes a single character and converts it into the corresponding ASCII value. For example, if we have "monkey" and call the function on each character, we should get the following ASCII values in order: `109, 111, 110, 107, 101, 121`

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

2. Checking strings for non-alphabetic characters

Write a function `contains_non_alpha` that takes a string and checks whether it contains any non-alphabetic characters. You can (and in fact should) use the built-in functions `ord` and `chr`.

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

- Did your program(s) produce error messages? Did it/they work, semantically? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task(s)? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

Give your programs meaningful names (ending in `.py`) and submit to Gradescope. For feedback, insert you printed program(s) here.

A6) Decoding binary patterns

- Binary to Decimal numbers (https://www.cuemath.com/numbers/binary-to-decimal/)

Write a function `decode_binary` that takes a string of `0`s and `1`s representing a binary number and returns the integer value of that binary number. For example, given the string `"110"` the function should return the integer `6`. Given `"00101001"`, it should return the integer `41`. Do not use the built-in function `int`.

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Did your program produce error messages? Did it work, semantically? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

Give your programs meaningful names (ending in `.py`) and submit to Gradescope.

For feedback, insert you printed program(s) here.

A7) Pig Latin

Pig Latin is a language game in which English words are altered to make them hard to understand by people not familiar with the rules. Write a function that takes an English word, translates that word into Pig Latin and **returns** the result. It should follow the below rules:

- If a word begins with a vowel, append "way" to the end of the word (e.g. `ear -> earway`)
- If a word begins with a consonant, remove all consonants from the beginning up to the first vowel and append them to the end of the word. Then append "ay" to the end of the word. (`pig -> igpay`, `star -> arstay`, `street -> eetstray`)
- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Did your program produce error messages? Did it work, semantically? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

Give your programs meaningful names (ending in `.py`) and submit to Gradescope.

For feedback, insert you printed program(s) here.

A8) Caesar Cipher

1. Caesar Cipher Encoding

A Caesar cipher (or shift cipher) encrypts text by "shifting" each letter a given number of positions in the alphabet. For example, if the number is `3`, then an `a` would become a `d`, `b` would become `e` and so on. At the end of the alphabet, we "loop back" to the beginning, so a `z` would become a `c`.

For example, if we encode the message "Caesar Ciphers are cool" with a shift of `7`, it becomes:

```
Jhlzhy Jpwolyz hyl jvvs.
```

If we encode that same message with a shift of `1`, it becomes:

```
Dbftbs Djqifst bsf dppm.
```

- Download the starter file `caesar.py` from the course website.
- Write a function called `encipher` that takes a string representing an unencoded message and a number indicating by how many positions each letter should be shifted. The function should return a new string which represents the corresponding Caesar Cipher encoding.

For this problem, you should assume that upper-case letters are encoded as upper-case letters, and all non-alphabetic characters are left unchanged. Some examples:

```
>>> encipher('xyza', 1)
'yzab'
>>> encipher('Z A', 1)
'A B'
>>> encipher("*ab?", 1)
'*bc?'
>>> encipher("This is a string!", 2)
'Vjku ku c uvtkpi!'
>>> encipher("Caesar cipher? I prefer Caesar salad.", 25)
'Bzdrzq bhogdq? H oqdedq Bzdrzq rzkzc.'
```

Feel free to add additional helper functions if you find it useful to do so.

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

2. Caesar Cipher Decoding:

- Write a function called `decipher` that takes a message encrypted with a Caesar Cipher and decodes it. The function should take two parameters - the encoded message and a number indicating by how many positions each character was shifted - and it should return the decoded message. For example:

```
>>> decipher('Bzdrzq bhogdq? H oqdedq Bzdrzq rzkzc.', 25)
'Caesar cipher? I prefer Caesar salad.'
```

Don't start a new file for this. Add your new function to the code that you already have in `caesar.py`.

Hint: the built-in functions `ord`, `chr`, and `isalpha` may be helpful for this assignment.

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Did your program produce error messages? Did it work, semantically? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

Give your programs meaningful names (ending in `.py`) and submit to Gradescope.

For feedback, insert you printed program(s) here.

A9) Written Questions

1. Convert the following numbers in binary representation to decimal representation and vice versa.

1011 …….. 34 ……

11010 …… 16 ……

01001 …… 354 ……

2. Translate the following binary addition problems into decimal representation. (Note the number of underscores is for spacing, it does not indicate the length of the answer.)

binary	decimal	binary	decimal	binary	decimal
110		0110		110	
	_______		_______		_______
+ 1	+_____	+100	+_____	+10	+_____
111		1010		1000	

3. Solve the following binary addition problems. That is, give answers (in binary). (Note the number of underscores is for spacing, it does not indicate the length of the answer.)

101	101	0101	0101	0100	010110001
+10	+01	+101	+011	+111	+11000111
_____	_____	_____	_____	_____	_____

4. A strategy often used when teaching addition in elementary school involves "carrying" the 10 when adding decimal numbers. For example, applying this strategy to 387 + 595 would look as follows:

```
 11
 387
+595
 982
```

Describe a similar strategy for adding binary numbers (i.e. give rules to determine which bits should be 1 and which should be 0 to represent the result). (Don't convert the binary numbers to decimal numbers).

……

……

……

……

……

5. Consider the following function definition when answering the questions below:

```
def not_a_good_function_name(aString):
    not_a_good_variable_name = ""
    for item in aString:
        not_a_good_variable_name+= item*2
    return not_a_good_variable_name
```

5.1 What will get printed to the screen when the function given above is called as follows: `print(not_a_good_function_name("6a7b2c"))`

.....

.....

5.2 In general words, what seems to be the purpose of this function?

.....

.....

.....

5.3 Suggest better names for the less than fantastic function and variable names used in this definition.

`not_a_good_function_name` .....

`not_a_good_variable_name` .....

A10) Caesar Cipher Code Breaking

If you haven't completed Ceasar Cipher Encoding/Decoding (A8) (aside from the optional parts), you'll need to do that first.

You will be submitting the following files:

- caesar.py

A Caesar cipher (or shift cipher) encrypts text by "shifting" each letter a given number of positions in the alphabet. For example, if the number is 3, then `a` would become `d`, `b` would become `e` and so on. At the end of the alphabet, we "loop back" to the beginning, so `z` would become a `c`.

For example, if we encode the message "Caesar Ciphers are cool" with a shift of `7`, it becomes:

```
Jhlzhy Jpwolyz hyl jvvs.
```

If we encode that same message with a shift of `1`, it becomes:

```
Dbftbs Djqifst bsf dppm.
```

In this section, you will add onto the program you wrote for the previous parts of the assignment. You should have written the functions encipher and decipher, which encode and decode a given string by shifting all letters by a given number of positions in the alphabet.

1. Open the file `caesar.py` which contains your solutions to the last assignment. Add all of your new code to this file. Your task is to write a function called `break_code` that decodes an encrypted message without knowing by how much the letters were shifted.

To do this your function will have to create all possible decodings, compare them and only keep the best one around, which gets returned in the end. For example:

```
>>> break_code('Bzdrzq bhogdq? H oqdedq Bzdrzq rzkzc.')
'Caesar cipher? I prefer Caesar salad.'
```

2. One strategy for determining which decoding is the best is to determine the probability of each candidate decoding by multiplying the letter probabilities for each letter in the decoding and to then choose the candidate decoding with the highest probability. But you are free to use a different strategy if you want to. Be sure to add a good comment to your decoding function explaining what strategy you used.

If you want to use the strategy based on letter probabilities, you can download a file called `letter_probabilities.py`. It provides a function that takes a letter and returns the probability of that letter in English text. Note that the implementation of that function uses dictionaries, which we have not yet discussed in class. However, you do not need to understand how this function is implemented in order to use it. All you need to know is that it is a function called `letter_prob` which takes one parameter (a character) and returns a probability. If the given character is a character from the English language, the function returns the probability of that letter. For all other characters it just returns `1`. To be able to use the function in your program, save the file `letter_probabilities.py` in the same folder as your file `caesar.py` and import it: `from letter_probabilities import letter_prob`

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Did your program produce error messages? Did it work, semantically? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

You don't have to and shouldn't modify the file `letter_probabilities.py`.

3. Note: some strings have more than one English deciphering and it is difficult to handle short strings correctly. Thus, your codebreaking function does not have to be perfect. But it should work for longer stretches of English text, e.g., sentences of 8+ words.

For feedback, insert you printed program(s) here.

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

B2: Share-out (optional)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....